Architecture

1. Architectural Overview

The proposed solution is a **scheduler-backed task watchdog** with a durable registry and a notification emitter.

```
OpenClaw agent starts background work

        |

        v

Background task registry (JSON state)

        |

        v

Scheduled watchdog (cron/systemd timer every 2–3 min)

        |

        +--> inspect task/session/process state

        +--> determine reminder / overdue / completion / failure

        |
```

```
        v

OpenClaw-native message/update emission

        |

        v

State/event log update
```

2. Major Components

2.1 Task Registry

- Durable JSON file under the OpenClaw workspace, e.g. `memory/background-tasks.json`.
- Stores active tasks, metadata, status, and last notification sent.

2.2 Watchdog Checker

- Local script or service that reads the registry.
- Looks up associated processes, sessions, or known task identifiers.
- Computes whether a notification is due.

2.3 Scheduler

- Preferred Phase 1 option: cron or systemd timer.
- Runs independently of agent conversational context.

2.4 Notification Emitter

- Uses OpenClaw-native routing to message the correct chat/session.
- Must include proof-oriented wording and the current observed state.

3. OpenClaw Ecosystem Fit

- The registry is local and durable, fitting the OpenClaw workspace model.
- The scheduler is external to ephemeral agent context, solving the memory problem.
- Notifications can route through OpenClaw messaging rather than custom chat plumbing.
- The system complements, but does not depend solely on, heartbeats.

4. Rationale

This architecture is preferred because it is:

- more reliable than “agent intention”
- simpler than a bespoke daemon framework
- better suited to strict provability than a purely conversational follow-up model